

Computación Paralela y Distribuida

2026-II

José Fiestas PhD

9 de septiembre de 2026

Universidad de Ingeniería y Tecnología
jfiestas@utec.edu.pe



Unidad 3: Comunicación y coordinación

Objetivos:

1. Pasos de Mensaje: MPI, Mensajes Punto a Punto, MPI, Comunicación Colectiva
2. Blocking vs non-blocking
3. Tipos MPI
4. Memoria Compartida: OMP, Constructores y cláusulas, CUDA, optimizacion con GPUs
5. Programacion Híbrida

Comunicación bloqueada (repaso)

Comunicación Punto a Punto - bloqueada

Sender mode	Notes	Synchronous?
Synchronous send	Message goes directly to receiver. Only completes when the receive has begun.	<i>synchronous</i>
Buffered send	Message is copied in to a "buffer" (provided by the application). Always completes (unless an error occurs), irrespective of receiver.	<i>asynchronous</i>
Standard send	Either synchronous or buffered (into a fixed size buffer provided by MPI system)	<i>both/hybrid</i>
Ready send	<i>Assumes</i> the matching receive has been called. Undefined behaviour (possibly an error) if the receiver is not ready.	<i>neither?</i>
Receive	Completes when a message has arrived	

Operation	MPI Call
Standard send	<code>MPI_Send</code>
Synchronous send	<code>MPI_Ssend</code>
Buffered send	<code>MPI_Bsend</code>
Ready send	<code>MPI_Rsend</code>
Receive	<code>MPI_Recv</code>

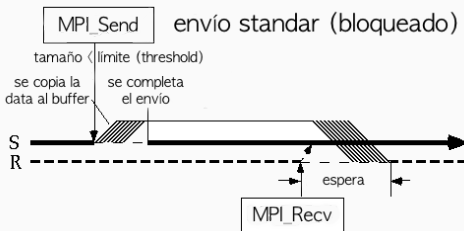
Comunicación bloqueada: standard

Si el **tamaño del mensaje es menor que un límite** (threshold) típicamente de algunos KB a MB

MPI_Send copia el mensaje en memoria (buffer) del proceso destino.

Entonces el proceso origen puede seguir operando.

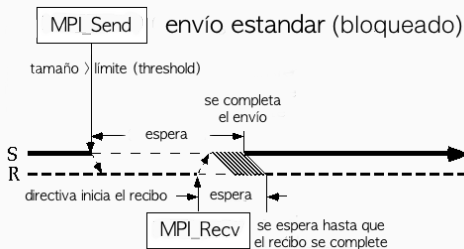
El buffer se copia al proceso destino cuando se llama a MPI_Recv



Comunicación bloqueada: standard

Si el **tamaño del mensaje es mayor que un límite (threshold)**, funciona como un envío sincrónico

De esta manera, mensajes pequeños tienen menor overhead (por el uso del buffer), mientras mensajes grandes no usan el buffer por el riesgo ocupar rápidamente todo el espacio.

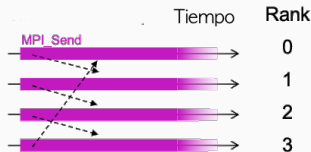
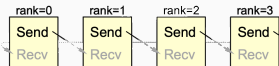


Ejemplo: Comunicación cíclica y no-cíclica

BLOQUEO MUTUO

```
MPI_Send(..., right_rank, ...)  
MPI_Recv(..., left_rank, ...)
```

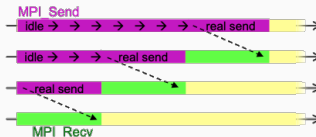
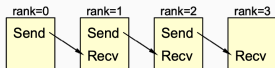
MPI comunica en forma sincrónica
MPI_Send espera a MPI_Recv



SECUENCIAL

```
if (myrank < size-1) MPI_Send(..., right_rank, ...);  
if (myrank > 0) MPI_Recv(..., left_rank, ...);
```

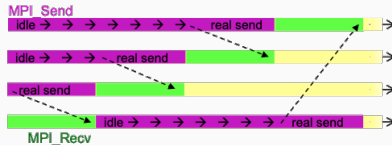
MPI comunica en forma sincrónica



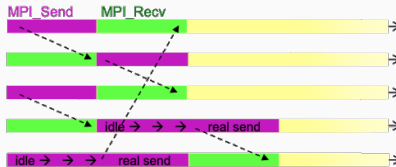
Ejemplo: Comunicación cíclica

SECUENCIAL

```
if (myrank < size-1) {  
    MPI_Send(..., right_rank, ...);  
    MPI_Recv(..., left_rank, ...);  
} else {  
    MPI_Recv(..., left_rank, ...);  
    MPI_Send(..., right_rank, ...);  
}
```



```
if (myrank%2 == 0) {  
    MPI_Send(..., right_rank, ...);  
    MPI_Recv(..., left_rank, ...);  
} else {  
    MPI_Recv(..., left_rank, ...);  
    MPI_Send(..., right_rank, ...);  
}
```



Comunicación no-bloqueada

Se realiza en 3 pasos:

- **Inicializa** comunicación no-bloqueada. La "I" significa "retorno inmediato" (local), ejecución independiente de otro proceso.
- Realiza **cómputo, o comunicación**
- Espera hasta que se **completa** la comunicación, i.e., el buffer de envío es leído, o el buffer de recibo se llena.

Send no-bloqueado:

Analogía: correos de invitación a un meeting (se confirma luego)

Receive no-bloqueado:

Analogía: Se reciben mensajes sin tener prendida la aplicación

Comunicación no bloqueada: standard

Si el **tamaño del mensaje es menor que un límite** (threshold) típicamente de algunos MB (eager protocol)

MPI_Isend

inicializa el envío

y retorna inmediatamente

para seguir operando.

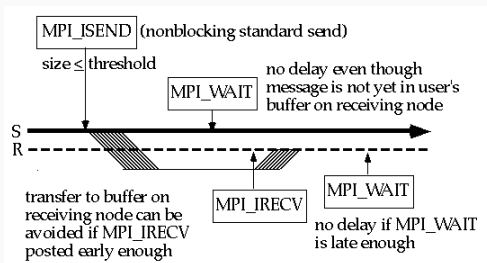
MPI_Wait se llama cuando

se necesita reutilizar

el buffer. MPI_Irecv

se ejecuta cuando el buffer

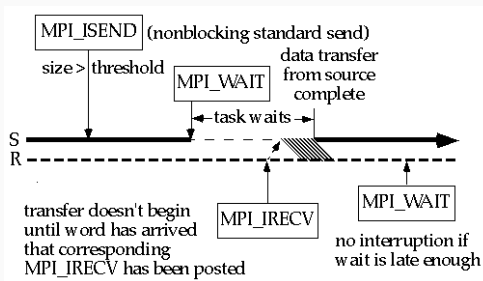
esta listo para recibir un mensaje. MPI_Wait se llama cuando se necesita utilizar el mensaje. Es conveniente declarar el MPI_Irecv antes de MPI_Isend. Esto permite evitar un bloqueo mutuo.



Comunicación no bloqueada: standard

Si el **tamaño del mensaje es mayor que un límite (threshold)** (rendezvous protocol)

El overhead se reduce en el período entre la llamada a MPI_Isend y MPI_Wait. Si MPI_Irecv se llama antes de MPI_Isend se reduce la sincronización.



Término de comunicación (wait, test)

```
int MPI_Wait(MPI_Request *req, MPI_Status *status)
```

Se llama cuando se requiere reutilizar el buffer (luego de un **MPI_Isend**), o cuando se requiere leer la data desde el buffer (luego de un **MPI_Irecv**), los cuales detienen su operación hasta que se completa la operación referenciada por Request ***req**

El estado de la operación esta referenciado por ***status**

```
int MPI_Test(MPI_Request *req, int *flag, MPI_Status *status)
```

Solo observa el fin de determinada operación. Flag es un puntero a entero que contendrá el resultado del test, que retorna TRUE o FALSE dependiendo del estado de la operación de comunicación.

MPI prueba el término de comunicación múltiple controlando todas (**all**) las comunicaciones, cualquiera (**any**) de ellas, o algunas (**some**) de ellas

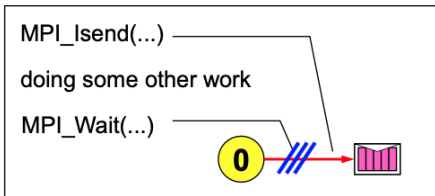
Test for completion	WAIT type (blocking)	TEST type (query only)
At least one, return exactly one	MPI_WAITANY	MPI_TESTANY
Every one	MPI_WAITALL	MPI_TESTALL
At least one, return all which completed	MPI_WAITSOME	MPI_TESTSOME

Comunicación no-bloqueada

El envío se inicia con **MPI_Isend**, y continúa con cómputo/comunicación

```
int MPI_Isend(void* buf, int count, MPI_Datatype datatype,  
int dest, int tag, MPI_Comm comm, MPI_Request *request)
```

- Nonblocking **send**



Isend + Wait = Send

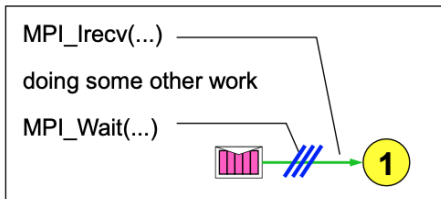
No utilice el buffer antes de completar la comunicación (i.e. entre Isend y Wait)

Comunicación no-bloqueada

El recibo se inicia con **MPI_Irecv**, y continúa con cómputo/comunicación

```
int MPI_Irecv(void* buf, int count, MPI_Datatype datatype,  
int source, int tag, MPI_Comm comm, MPI_Request *request)
```

- Nonblocking **receive**

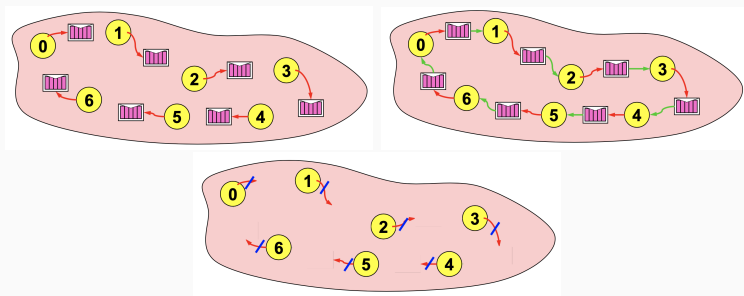


Irecv + Wait = Recv

No utilice el buffer antes de completar la comunicación (i.e. entre Irecv y Wait)

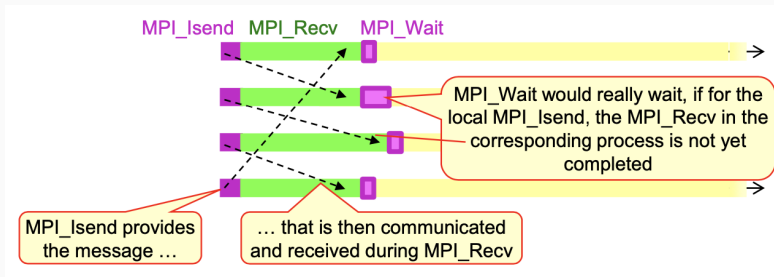
Caso 1: anillo ciclico, envío no-bloqueado

Se **inicializa el envío** (no bloqueado, al vecino derecho). Se **continúa trabajando** (aquí, recibiendo mensaje del vecino izquierdo).



La comunicación se **completa con MPI_Wait** del envío no bloqueado

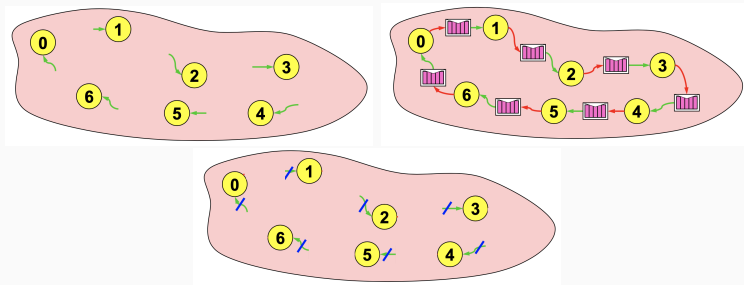
Caso 1: anillo ciclico, envío no-bloqueado



Comunicación **no es secuencial ni existe bloqueo mutuo**

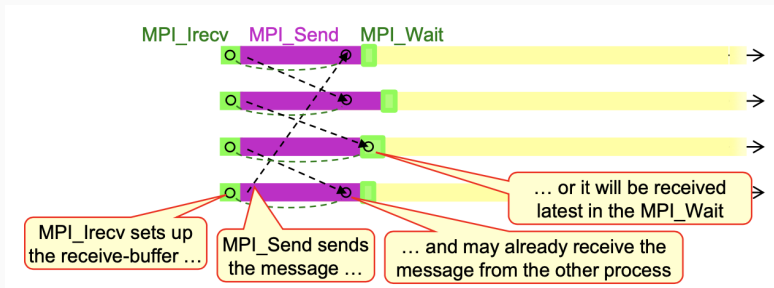
Caso 2: anillo ciclico, recibo no-bloqueado

Se **inicializa el recibo** (no bloqueado, del vecino izquierdo). Se **continua trabajando** (aquí, enviando mensaje al vecino derecho).



La comunicación se **completa con MPI_Wait** del recibo no bloqueado

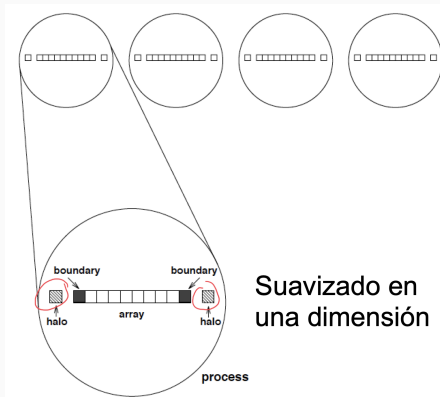
Caso 2: anillo ciclico, recibo no-bloqueado



Comunicación **no** es secuencial ni existe bloqueo mutuo

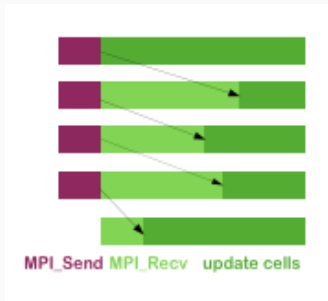
Caso 2: Suavizado

Operación de suavizado
(**smoothing**) promedia
los vecinos inmediatos.
Los extremos del sub-array
necesitan comunicar sus
valores a procesos vecinos



Caso 3: Suavizado (bloqueo)

La comunicación no retorna ningún valor hasta que ésta esté terminada (cuando el buffer es libre de nuevo).



Caso 3: Suavizado (bloqueo), PRAM por proceso

Algorithm 1: Suavizado (bloqueo)

Input: array A de $n/p + 2$ elementos, $1 \leq j \leq p$ (p es total de procesos)

Output: array A de n/p elementos con smoothing

for $i = 1$ **to** n/p **do**

$A(i) := \text{smooth}(i - 1, i, i + 1)$

if $(j == 1)$ **then**

$\text{Send}(A(n/p) \rightarrow j + 1), \text{Receive}(A(1) \leftarrow j + 1)$

else

if $(j == p)$ **then**

$\text{Send}(A(1) \rightarrow j - 1), \text{Receive}(A(n/p) \leftarrow j - 1)$

end if

end if

else

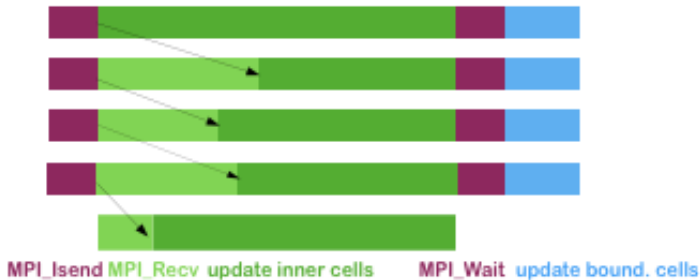
$\text{Send}(A(1) \rightarrow j - 1), \text{Send}(A(n/p) \rightarrow j + 1)$

$\text{Receive}(A(n/p) \leftarrow j - 1), \text{Receive}(A(1) \leftarrow j + 1)$

end if

Caso 2: Suavizado (no-bloqueo)

Se necesita separar el envío y recibo de información por los vecinos. Para ello, el `Irecv` retorna valores antes de completarse la comunicación, permitiendo hacerse cargo de otros envíos/recibos/operaciones.



Caso 2: Suavizado (no bloqueo), PRAM por proceso

Algorithm 2: Suavizado (no bloqueo)

Input: array A de $n/p + 2$ elementos, $1 \leq j \leq p$ (p es total de procesos)

Output: array A de n/p elementos con smoothing

$A(1) := \text{smooth}(0, 1, 2)$

$A(n/p) := \text{smooth}((n/p - 1), (n/p), (n/p + 1))$

if ($j == 1$) **then**

| $\text{l send}(A(n/p) \rightarrow j + 1), \text{l receive}(A(1) \leftarrow j + 1)$

end if

else

| **if** ($j == p$) **then**

| | $\text{l send}(A(1) \rightarrow j - 1), \text{l receive}(A(n/p) \leftarrow j - 1)$

| **end if**

end if

else

| $\text{l send}(A(1) \rightarrow j - 1), \text{l send}(A(n/p) \rightarrow j + 1)$

| $\text{l receive}(A(n/p) \leftarrow j - 1), \text{l receive}(A(1) \leftarrow j + 1)$

end if

Caso 2: Suavizado (no bloqueo), PRAM por proceso

Algorithm 3: Suavizado (no bloqueo)

Input: array A de $n/p + 2$ elementos, $1 \leq j \leq p$ (p es total de procesos)

Output: array A de n/p elementos con smoothing

```
  for  $i = 2$  to  $n/p - 1$  do
    |    $A(i) := \text{smooth}(i - 1, i, i + 1)$ 
  end for
  MPI_Wait()
```

Cada llamada no-bloqueada necesita un **request** y **MPI_Wait()**

```
MPI_Request request;  
MPI_Isend(buf,count,datatype,  
dest,tag,comm,&request);  
MPI_Wait(&request,&status);
```

```
MPI_Request request;  
MPI_Irecv(buf,count,datatype,  
src,tag,comm,&request);  
MPI_Wait(&request,&status);
```

- Send y Recv

pueden ser bloqueados
o no-bloqueados

-

Un Send bloqueado puede
ser usado con un Recv
no- bloqueado, y viceversa

- Send no-bloqueados
pueden usar cualquier
modo (sincronizado, buffered, standard, o ready)

Communication Modes

Non-Blocking Operation	MPI Call
Standard send	MPI_Isend
Synchronous send	MPI_Issend
Buffered Send	MPI_Ibsend
Ready send	MPI_Irsend
Receive	MPI_Irecv

slido

Join at
slido.com
#2022 7169





<https://www.mpich.org/documentation/guides/>



Peter S. Pacheco. *An Introduction to Parallel Programming*. 1st. Morgan Kaufmann, 2011. isbn: 978-0-12-374260- 5 Ch 3



Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP*. 1st. McGraw-Hill Education Group, 2003. isbn: 0071232656 Ch 4

